

SOLUSI ACUAN FASILITATOR

Kontainerisasi Laravel hingga Produksi di Kubernetes

Aspek	Keterangan
Durasi	4 hari ($\pm$ 24 jam efektif)
Tingkat	Menengah
Jumlah latihan	21 latihan mandiri
Peran dokumen	Kunci jawaban — JANGAN dibagikan ke peserta

Daftar Isi

Daftar Isi.....	2
Cara Memakai Dokumen Ini.....	5
Struktur Tiap Solusi	5
Cara Membaca Perintah	5
Hari 1 — Fondasi: dari Aplikasi Menjadi Image	6
L1.1 Mengurutkan Ulang Dockerfile	6
Langkah Penyelesaian	7
Jawaban Acuan	7
Pembahasan.....	7
L1.2 Membaca Pembagian Hak Akses PHP-FPM.....	8
Langkah Penyelesaian	8
Jawaban Acuan	8
Pembahasan.....	8
L1.3 Membuktikan Perkakas Build Tidak Ikut Terbawa	9
Langkah Penyelesaian	9
Jawaban Acuan	9
Pembahasan.....	9
L1.4 Menemukan Berkas yang Seharusnya Tidak Ikut.....	10
Langkah Penyelesaian	10
Jawaban Acuan	10
Pembahasan.....	10
Hari 2 — Stack Lengkap dengan Docker Compose	11
L2.1 Menambah Service Baru ke Stack	11
Langkah Penyelesaian	11
Jawaban Acuan	12
Pembahasan.....	12
L2.2 Membedakan Container "Up" dan Database "Siap"	12
Langkah Penyelesaian	12
Jawaban Acuan	13
Pembahasan.....	13
L2.3 Menentukan Kebijakan Eviction.....	13
Langkah Penyelesaian	13
Jawaban Acuan	14

Pembahasan.....	14
L2.4 Melacak Job yang Gagal	14
Langkah Penyelesaian	15
Jawaban Acuan	15
Pembahasan.....	15
L2.5 Cache Konfigurasi di Waktu yang Salah	15
Langkah Penyelesaian	16
Jawaban Acuan	16
Pembahasan.....	16
L2.6 Menguji Paritas Dua Web Server	17
Langkah Penyelesaian	17
Jawaban Acuan	17
Pembahasan.....	17
L2.7 Mengukur Dampak Penempatan vendor/	18
Langkah Penyelesaian	18
Jawaban Acuan	18
Pembahasan.....	19
Hari 3 — Kubernetes: Konsep Inti.....	19
L3.1 Menelusuri Deployment sampai Endpoint.....	19
Langkah Penyelesaian	19
Jawaban Acuan	19
Pembahasan.....	20
L3.2 Perubahan Konfigurasi yang Tidak Berlaku	20
Langkah Penyelesaian	20
Jawaban Acuan	20
Pembahasan.....	21
L3.3 Menguji Ketahanan Data.....	21
Langkah Penyelesaian	21
Jawaban Acuan	22
Pembahasan.....	22
L3.4 Memilih Probe yang Tepat	22
Langkah Penyelesaian	22
Jawaban Acuan	23
Pembahasan.....	23

L3.5 Membuat Overlay Staging.....	23
Langkah Penyelesaian	23
Jawaban Acuan	24
Pembahasan.....	24
Hari 4 — Produksi dan Keamanan	25
L4.1 Memperbaiki Manifest yang Ditolak.....	25
Langkah Penyelesaian	25
Jawaban Acuan	25
Pembahasan.....	26
L4.2 Menemukan Direktori yang Perlu Writable	26
Langkah Penyelesaian	26
Jawaban Acuan	27
Pembahasan.....	27
L4.3 Menguji Apakah NetworkPolicy Benar-benar Ditegakkan	27
Langkah Penyelesaian	27
Jawaban Acuan	28
Pembahasan.....	28
L4.4 Mengukur Kegagalan Request Selama Rollout	28
Langkah Penyelesaian	29
Jawaban Acuan	29
Pembahasan.....	29
L4.5 Analisis Akar Masalah dari Log	30
Langkah Penyelesaian	30
Jawaban Acuan	30
Pembahasan.....	31
Penutup.....	31

Dokumen ini memuat jawaban acuan untuk seluruh latihan di Panduan Peserta. Ia dipegang fasilitator dan TIDAK dibagikan ke peserta selama latihan berlangsung.

Struktur Tiap Solusi

Cara Membaca Perintah

```
docker run --rm --entrypoint sh laravel-app:latest -c "command -v composer"
```

| | | | | |

| | | | | +-- argumen perintah

| | | | +-- image yang dijalankan

| | | +-- ganti entrypoint bawaan image

| | +-- hapus container setelah selesai

| +-- sub-perintah

+-- program utama

© 2026 INIXINDO — Solusi Acuan — Kontainerisasi Laravel hingga Produksi di Kubernetes

-n <ns>	Namespace yang dituju	seluruh perintah kubectl
-l <label>	Saring berdasarkan label	kubectl get, logs, delete
--previous	Log container SEBELUM restart terakhir	kubectl logs saat diagnosis
-o jsonpath	Ambil satu field tertentu dari objek	kubectl get untuk skrip
--dry-run=client	Susun objek tanpa mengirim ke server	menguji manifest
-v	Volume: petakan direktori atau named volume	docker run, docker compose

CARA MENILAI

Nilai penalarannya, bukan kesamaan kata. Banyak pertanyaan menerima lebih dari satu jawaban benar; bagian Pembahasan menyebutkan variasi yang tetap diterima.

Untuk pertanyaan yang meminta angka hasil pengukuran, yang dinilai adalah peserta benar-benar mengukur dan membaca angkanya — bukan ketepatan angkanya, yang memang berbeda tiap mesin.

Untuk pertanyaan yang meminta DUA hal (mis. solusi dan alasannya), jawaban yang hanya menyebut satu bernilai separuh. Ini ditandai eksplisit di bagian Pembahasan.

Bila peserta menemukan jalur penyelesaian berbeda yang tetap membuktikan hal yang sama, hargai — kemampuan menyusun pengujian sendiri adalah tujuan sebenarnya.

Hari 1 — Fondasi: dari Aplikasi Menjadi Image

LATIHAN HARI 1

- L1.1 — Mengurutkan Ulang Dockerfile
- L1.2 — Membaca Pembagian Hak Akses PHP-FPM
- L1.3 — Membuktikan Perkakas Build Tidak Ikut Terbawa
- L1.4 — Menemukan Berkas yang Seharusnya Tidak Ikut

L1.1 Mengurutkan Ulang Dockerfile

Soal ringkas: Sebuah tim mengeluh: setiap kali mereka mengubah satu baris kode PHP, build image memakan waktu lebih dari empat menit karena Composer mengunduh ulang seluruh dependensi.

Konsep yang dilatih: Layer image · Cache build · Urutan instruksi Dockerfile

Kode yang diberikan di lembar soal

```
FROM php:8.4-fpm-alpine
WORKDIR /var/www/html
```

```
COPY src/ ./
```

```
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
RUN composer install --no-dev --prefer-dist
```

Langkah Penyelesaian

- [1] Bangun versi awal dua kali, catat waktunya
 Measure-Command { docker build -t uji:awal -f Dockerfile.awal . }
 Measure-Command { docker build -t uji:awal -f Dockerfile.awal . }
- [2] Ubah satu berkas kode, lalu bangun lagi
 Add-Content src/routes/web.php "// uji cache"
 Measure-Command { docker build -t uji:awal -f Dockerfile.awal . }
- [3] Perbaiki urutan: manifest dependensi disalin dan dipasang LEBIH DULU
 (lihat Jawaban Acuan)
- [4] Ulangi langkah [1] dan [2] pada versi yang sudah diperbaiki

Yang harus terlihat:

- Sebelum perbaikan: build setelah perubahan kode memakan waktu hampir sama dengan build pertama, dan baris `composer install` TIDAK menampilkan CACHED.
- Sesudah perbaikan: build setelah perubahan kode selesai dalam hitungan detik, dan baris `composer install` menampilkan CACHED.

Jawaban Acuan

L1.1.1 Tulis ulang urutan instruksi yang benar.

Manifest dependensi disalin dan dipasang lebih dulu, kode aplikasi menyusul:

```
FROM php:8.4-fpm-alpine
WORKDIR /var/www/html
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
COPY src/composer.json src/composer.lock ./
RUN composer install --no-dev --prefer-dist
COPY src/ ./
```

L1.1.2 Berapa detik build kedua SEBELUM perbaikan, dan berapa SESUDAH perbaikan?

Angkanya bergantung mesin. Yang dinilai adalah POLANYA: sebelum perbaikan waktunya tetap besar (puluhan detik sampai menit), sesudah perbaikan turun drastis menjadi hitungan detik.

L1.1.3 Instruksi mana yang paling mahal, dan kenapa posisinya menentukan?

`RUN composer install` yang paling mahal. Karena cache sebuah layer ikut batal bila ada layer di bawahnya yang berubah, menaruh `COPY src/ ./` di atasnya membuat baris composer selalu dijalankan ulang setiap kali ada satu karakter kode yang berubah.

Pembahasan

- Kesalahan umum: memindahkan `COPY src/ ./` ke bawah tetapi lupa menyalin `composer.json` dan `composer.lock` lebih dulu. Build akan gagal karena Composer tidak menemukan manifest.
- Kesalahan umum: menambahkan `--no-cache` untuk "memastikan bersih". Itu justru mematikan satu-satunya mekanisme yang mempercepat build.
- Diterima juga: menambahkan cache mount BuildKit (`RUN --mount=type=cache,target=/tmp/composer`). Ini bahkan lebih baik, asalkan urutan instruksinya tetap dibetulkan lebih dulu.

L1.2 Membaca Pembagian Hak Akses PHP-FPM

Soal ringkas: Jalankan sebuah container php-fpm, lalu amati daftar proses di dalamnya.

Konsep yang dilatih: Arsitektur PHP-FPM · Drop privilege · Prinsip hak akses minimal

Langkah Penyelesaian

```
[1] Jalankan container php-fpm di latar belakang
    docker run -d --name coba-fpm php:8.4-fpm-alpine

[2] Lihat daftar proses beserta pemiliknya
    docker exec coba-fpm ps -o user,args
        ps      = daftar proses
        -o      = pilih kolom yang ditampilkan
        user    = pemilik proses; args = perintah lengkapnya

[3] Bersihkan
    docker rm -f coba-fpm
```

Yang harus terlihat:

- Satu baris `php-fpm: master process` milik root.
- Beberapa baris `php-fpm: pool www` milik www-data.

Jawaban Acuan

L1.2.1 Salin dua baris pertama keluaran daftar proses.

Contoh keluaran:

USER	COMMAND
root	php-fpm: master process (/usr/local/etc/php-fpm.conf)
www-data	php-fpm: pool www

L1.2.2 User apa yang menjalankan proses master, dan user apa yang menjalankan pekerja?

Master dijalankan root; pekerja dijalankan www-data (uid 82 pada image Alpine).

L1.2.3 Kenapa master perlu hak akses yang lebih tinggi daripada pekerja?

Master perlu root hanya untuk dua hal di awal: membuka socket/port dan membaca berkas konfigurasi milik root. Sesudah itu ia menurunkan hak akses pekerja ke user biasa.

L1.2.4 Apa risikonya bila SEMUA proses dijalankan sebagai root?

Kode aplikasi dieksekusi oleh PEKERJA. Bila pekerja berjalan sebagai root, satu celah pada kode aplikasi (mis. unggahan berkas yang tidak divalidasi) langsung memberi akses root di dalam container — termasuk kemampuan menulis ulang kode aplikasi itu sendiri.

Pembahasan

- Kesalahan umum: menjawab "supaya rapi". Alasannya bukan kerapian, melainkan membatasi kerusakan bila kode aplikasi dieksploitasi.
- Diterima juga: peserta yang menyebut bahwa di Kubernetes bahkan master pun dijalankan non-root karena Pod Security restricted melarang root sama sekali. Ini justru menunjukkan peserta membaca lebih jauh.

L1.3 Membuktikan Perkakas Build Tidak Ikut Terbawa

Soal ringkas: Sebuah image produksi diklaim "sudah multi-stage dan bersih".

Konsep yang dilatih: Multi-stage build · Permukaan serangan image · Verifikasi klaim

Langkah Penyelesaian

```
[1] Bangun stage vendor dan image produksi secara terpisah
docker build -t coba-vendor -f docker/php/Dockerfile --target vendor .
docker build -t laravel-app:latest -f docker/php/Dockerfile --target app .

[2] Periksa isi image produksi TANPA melewati entrypoint
docker run --rm --entrypoint sh laravel-app:latest \
  -c "command -v composer || echo TIDAK ADA"
  --entrypoint sh = ganti entrypoint bawaan dengan shell
  -c              = jalankan satu perintah lalu keluar

[3] Ulangi untuk npm dan .env
docker run --rm --entrypoint sh laravel-app:latest \
  -c "command -v npm || echo TIDAK ADA"
docker run --rm --entrypoint sh laravel-app:latest \
  -c "ls -a /var/www/html | grep -c env"

[4] Bandingkan ukuran
docker images | Select-String "coba-vendor|laravel-app"
```

Yang harus terlihat:

- Langkah [2] dan [3] menjawab TIDAK ADA untuk composer dan npm.
- Pemeriksaan .env mengembalikan angka 0.
- Image produksi berukuran jauh lebih kecil daripada stage vendor.

Jawaban Acuan

L1.3.1 Perintah apa yang Anda pakai untuk memeriksanya?

`docker run --rm --entrypoint sh <image> -c "command -v composer"` dan seterusnya, ditambah `docker images` untuk ukuran serta `docker history` untuk riwayat layer.

L1.3.2 Apakah Composer, npm, dan .env ditemukan di dalam image?

Ketiganya TIDAK ditemukan. Composer dan npm tinggal di stage vendor/assets; .env dikecualikan lewat .dockerignore dan dihapus di stage app.

L1.3.3 Berapa selisih ukuran antara stage vendor dan image produksi?

Angkanya bergantung versi. Pada stack acuan: image produksi 272 MB, sementara stage vendor jauh lebih besar karena membawa Composer beserta cache-nya.

L1.3.4 Kenapa memeriksa dengan `docker images` saja tidak cukup untuk membuktikan klaim ini?

`docker images` hanya menunjukkan ukuran, bukan ISI. Image kecil tetap bisa memuat berkas rahasia, dan image besar belum tentu memuat perkakas build. Klaim tentang isi hanya bisa dibuktikan dengan memeriksa isinya.

Pembahasan

- Kesalahan umum: menjalankan `docker run <image> composer --version` tanpa `--entrypoint`. Entrypoint aplikasi akan berjalan lebih dulu dan gagal karena APP_KEY belum diset, sehingga peserta salah menyimpulkan penyebabnya.

- Poin penting yang layak dihargai: peserta yang menyadari bahwa pertanyaan terakhir adalah inti latihan ini — ukuran bukan bukti isi.

L1.4 Menemukan Berkas yang Seharusnya Tidak Ikut

Soal ringkas: Sebuah proyek dibangun dengan `.dockerignore` yang hanya berisi satu baris: `.git`.

Konsep yang dilatih: Build context · `.dockerignore` · Kebocoran rahasia lewat image

Langkah Penyelesaian

```
[1] Bangun dengan .dockerignore seadanya, catat ukuran context
    docker build -f docker/php/Dockerfile --target base -t uji .
    (perhatikan baris "transferring context")

[2] Lengkapi .dockerignore (lihat Jawaban Acuan)

[3] Bangun ulang dan bandingkan ukuran context

[4] Pastikan rahasia tidak ada di dalam image
    docker run --rm --entrypoint sh laravel-app:latest \
      -c "ls -la /var/www/html | grep -c env"
```

Yang harus terlihat:

- Ukuran context turun drastis, biasanya dari ratusan megabyte menjadi satuan megabyte.
- Pemeriksaan `.env` mengembalikan angka 0.

Jawaban Acuan

L1.4.1 Sebutkan minimal lima entri yang perlu ditambahkan ke `.dockerignore`.

Minimal: `.env` dan `.env.*` (rahasia); `src/vendor` dan `src/node_modules` (dibangun ulang di stage tersendiri); `src/public/build` dan `src/public/hot` (artefak Vite); `src/storage/logs/*` serta `src/bootstrap/cache/*` (artefak runtime); `.git` (riwayat).

L1.4.2 Untuk berkas `.env`: apa akibat konkret bila ia ikut masuk ke image?

Image umumnya dikirim ke registry dan bisa ditarik banyak orang. Siapa pun yang bisa menarik image itu dapat membaca seluruh isinya, termasuk password database dan `APP_KEY`. Mengganti password setelah bocor pun tidak menghapus image lama yang sudah beredar.

L1.4.3 Untuk `public/hot`: apa akibat konkret bila ia ikut masuk ke image produksi?

Berkas `hot` dibuat Vite saat mode pengembangan dan berisi alamat dev server (`http://localhost:5173`). Bila ikut ke produksi, Laravel menganggap dev server sedang berjalan dan memuat seluruh aset dari alamat itu — di server, seluruh CSS dan JavaScript gagal dimuat dan tampilan rusak total.

L1.4.4 Berapa ukuran build context sebelum dan sesudah perbaikan Anda?

Angkanya bergantung proyek. Yang dinilai adalah adanya penurunan signifikan dan peserta benar-benar membaca angkanya dari keluaran build, bukan menebak.

Pembahasan

- Kesalahan umum: hanya menyebut `vendor` dan `node_modules` karena fokus pada ukuran, lalu melewati `.env`. Ukuran hanyalah efek samping; kebocoran rahasia adalah risiko utamanya.

- Kesalahan umum: menambahkan `src/storage` seluruhnya, sehingga berkas `.gitignore` penanda struktur direktori ikut hilang dan entripoint harus membuat ulang semuanya.
- Diterima juga: peserta yang menambahkan `**/.DS_Store`, `Thumbs.db`, atau berkas IDE. Tidak wajib, tetapi menunjukkan pemahaman yang benar.

Hari 2 — Stack Lengkap dengan Docker Compose

LATIHAN HARI 2

- L2.1 — Menambah Service Baru ke Stack
- L2.2 — Membedakan Container "Up" dan Database "Siap"
- L2.3 — Menentukan Kebijakan Eviction
- L2.4 — Melacak Job yang Gagal
- L2.5 — Cache Konfigurasi di Waktu yang Salah
- L2.6 — Menguji Paritas Dua Web Server
- L2.7 — Mengukur Dampak Penempatan vendor/

L2.1 Menambah Service Baru ke Stack

Soal ringkas: Tim ingin menambahkan Mailpit — server SMTP palsu untuk menangkap email saat pengembangan — ke stack Compose yang sudah ada.

Konsep yang dilatih: Service Compose · Resolusi nama antar service · Konfigurasi lewat environment

Langkah Penyelesaian

- [1] Tambahkan service ke `compose.override.yaml` (khusus pengembangan)


```
mailpit:
  image: axllent/mailpit:latest
  restart: unless-stopped
  ports:
    - "8025:8025"          # antarmuka web
  networks: [backend]     # WAJIB: satu network dengan app
```
- [2] Arahkan Laravel ke service itu (`.env`)


```
MAIL_MAILER=smtp
MAIL_HOST=mailpit
MAIL_PORT=1025
```
- [3] Terapkan


```
docker compose up -d
```
- [4] Kirim email percobaan


```
docker compose exec app php artisan tinker \
  --execute="Mail::raw('uji', fn($m) => $m->to('a@b.c')->subject('Uji'));"
```
- [5] Buka `http://localhost:8025`

Yang harus terlihat:

- Service mailpit berstatus Up pada `docker compose ps`.
- Email percobaan muncul di antarmuka web Mailpit.

Jawaban Acuan

L2.1.1 Tulis blok YAML service yang Anda tambahkan.

Blok minimal yang benar memuat image, port 8025 yang dipublikasikan, dan — yang paling sering terlupa — `networks: [backend]` agar berada di jaringan yang sama dengan aplikasi.

L2.1.2 Variabel environment apa saja yang perlu diubah pada aplikasi?

`MAIL_MAILER=smtp`, `MAIL_HOST=mailpit`, `MAIL_PORT=1025`. `MAIL_HOST` harus berisi nama service.

L2.1.3 Bagaimana Anda membuktikan email benar-benar tertangkap?

Dengan mengirim email lewat tinker lalu melihatnya muncul di `http://localhost:8025`. Menerima jawaban lain yang setara, mis. memeriksa log mailpit.

L2.1.4 Kenapa `MAIL_HOST` tidak boleh diisi `'localhost'`?

Di dalam container, `localhost` menunjuk ke container ITU SENDIRI — bukan ke mesin host dan bukan ke container lain. Koneksi akan ditolak karena tidak ada yang mendengarkan di port 1025 di dalam container aplikasi.

Pembahasan

- Kesalahan paling sering: lupa `networks: [backend]`. Service tetap menyala, tetapi aplikasi tidak bisa me-resolve namanya dan gagal dengan error koneksi.
- Kesalahan umum: mempublikasikan port 1025 ke host. Tidak diperlukan — yang menghubunginya adalah container lain di jaringan yang sama, bukan browser.
- Diterima juga: menaruh mailpit di `compose.yaml` dengan profil tersendiri, asalkan peserta bisa menjelaskan kenapa ia tidak pantas ikut ke produksi.

L2.2 Membedakan Container "Up" dan Database "Siap"

Soal ringkas: Hapus seluruh stack beserta volumenya, lalu jalankan ulang sambil mengamati dengan cermat kapan container database berstatus Up dan kapan ia benar-benar siap menerima koneksi dari aplikasi.

Konsep yang dilatih: `Healthcheck` · `depends_on: service_healthy` · Inisialisasi MySQL

Langkah Penyelesaian

```
[1] Bersihkan total, termasuk volume
    docker compose down -v

[2] Jalankan hanya database, amati statusnya berubah
    docker compose up -d mysql
    docker compose ps
    (ulangi beberapa kali; perhatikan (health: starting) lalu (healthy))

[3] Baca penanda inisialisasi di log
    docker compose logs mysql | \
        Select-String "Initializing database|Temporary server|initialized"

[4] Jalankan sisanya, amati entripoint aplikasi menunggu
    docker compose up -d
    docker compose logs app | Select-String "entrypoint"
```

Yang harus terlihat:

- Status berpindah dari `(health: starting)` ke `(healthy)` setelah beberapa puluh detik.

- Log memuat berurutan: "Initializing database files", "Temporary server started", "Database files initialized".
- Log aplikasi memuat "Menunggu database ..." lalu "Database siap."

Jawaban Acuan

L2.2.1 Berapa selisih waktu antara container berstatus Up dan berstatus healthy?

Umumnya 20–60 detik pada inisialisasi pertama, dan jauh lebih singkat pada start berikutnya karena inisialisasi tidak diulang. Angka pastinya bergantung mesin.

L2.2.2 Tiga baris apa di log MySQL yang menandai inisialisasi berjalan lengkap?

"Initializing database files", "Temporary server started", dan "Database files initialized". Ketiganya hanya muncul saat direktori data masih kosong.

L2.2.3 Dua mekanisme apa yang mencegah aplikasi menyambung sebelum database siap?

Pertama, `healthcheck` pada service mysql dipadukan dengan `depends_on: condition: service_healthy` pada aplikasi. Kedua, entypoint aplikasi sendiri menunggu port database benar-benar bisa dihubungi sebelum melanjutkan.

L2.2.4 Apa yang terjadi bila proses inisialisasi terputus di tengah?

Direktori data berisi tabel sistem tetapi TANPA user aplikasi. Karena direktori sudah tidak kosong, MySQL melewati inisialisasi pada start berikutnya dan berjalan normal selamanya — sementara aplikasi ditolak dengan pesan "Host ... is not allowed to connect" yang menyesatkan. Menghapus container tidak memperbaikinya; volumenya yang harus dikosongkan.

Pembahasan

- Kesalahan umum: menjawab bahwa `depends_on` saja sudah cukup. Tanpa `condition: service_healthy`, ia hanya menunggu container DIMULAI.
- Poin penting: peserta yang menyadari bahwa healthcheck berbasis `mysqladmin ping` lemah karena keluar dengan status sukses walaupun autentikasi ditolak, layak dihargai — itu persis akar masalah pada pertanyaan keempat.

L2.3 Menentukan Kebijakan Eviction

Soal ringkas: Untuk masing-masing kasus di bawah, tentukan kebijakan `maxmemory-policy` yang tepat dan jelaskan akibatnya bila salah pilih.

Konsep yang dilatih: Kebijakan eviction Redis · Pemisahan beban kerja · Ketahanan data

Langkah Penyelesaian

```
[1] Periksa kebijakan yang berlaku pada masing-masing instance
docker compose exec redis sh -c 'redis-cli --no-auth-warning \
  -a "$REDIS_PASSWORD" config get maxmemory-policy'
docker compose exec redis-cache sh -c 'redis-cli --no-auth-warning \
  -a "$REDIS_PASSWORD" config get maxmemory-policy'

[2] Buktikan pemisahannya benar-benar bekerja
docker compose exec app php artisan tinker \
  --execute="Cache::put('uji','ok',60);"
docker compose exec redis-cache sh -c 'redis-cli -n 1 keys ""'
docker compose exec redis sh -c 'redis-cli -n 0 keys "laravel_horizon*"'
```

Yang harus terlihat:

- Instance redis menjawab `noeviction`; instance redis-cache menjawab `allkeys-lru`.
- Kunci cache hanya muncul di redis-cache, kunci Horizon hanya di redis.

Jawaban Acuan

L2.3.1 Kasus A — kebijakan apa, dan alasannya?

`allkeys-lru`. Data cache bisa dibangun ulang dari sumber aslinya, jadi membuang entri lama saat memori penuh justru perilaku yang diinginkan.

L2.3.2 Kasus B — kebijakan apa, dan apa akibatnya bila salah?

`noeviction`. Bila memakai `allkeys-lru`, job invoice yang belum diproses bisa dibuang begitu saja — tanpa error, tanpa log, dan tanpa ada yang tahu invoice itu tidak pernah terkirim.

L2.3.3 Kasus C — kebijakan apa, dan apa yang dialami pengguna bila salah?

`noeviction`. Bila sesi dibuang, pengguna tiba-tiba ter-logout di tengah pekerjaan tanpa sebab yang bisa dijelaskan, dan keluhannya sulit direproduksi.

L2.3.4 Kasus D — apa saran Anda, dan kenapa memisahkan nomor database tidak menyelesaikannya?

Pisahkan menjadi dua instance: satu untuk cache dengan `allkeys-lru`, satu untuk queue dan session dengan `noeviction`. Memisahkan nomor database TIDAK menyelesaikan apa pun, karena kebijakan eviction dan batas memori berlaku pada tingkat server — bukan per-database.

Pembahasan

- Kesalahan umum: menjawab `volatile-lru` untuk kasus B dengan alasan "job kan punya TTL". Job pada antrean umumnya tidak diberi TTL, dan menggantung keselamatan data pada ada tidaknya TTL adalah asumsi yang rapuh.
- Diterima juga untuk kasus C: memindahkan sesi ke database bila peserta bisa menjelaskan konsekuensi kinerjanya.
- Cara menilai kasus D: jawaban lengkap harus menyebut DUA hal — solusinya (dua instance) DAN alasan kenapa pemisahan database tidak cukup. Menyebut satu saja bernilai separuh.

L2.4 Melacak Job yang Gagal

Soal ringkas: Buat sebuah job yang sengaja selalu gagal, kirim ke antrean, lalu lacak kegagalannya sampai ke pesan kesalahan aslinya.

Konsep yang dilatih: Siklus hidup job · Percobaan ulang · Diagnosis lewat log dan dashboard

Kode yang diberikan di lembar soal

```
<?php

namespace App\Jobs;

use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Queue\Queueable;

class JobGagal implements ShouldQueue
{
    use Queueable;

    public function handle(): void
    {
        throw new \RuntimeException('Sengaja gagal untuk latihan.');
```

```
}
```

Langkah Penyelesaian

- [1] Buat berkas `src/app/Jobs/JobGagal.php` (isi ada di lembar soal)
- [2] Kirim ke antrean

```
docker compose exec app php artisan tinker \
  --execute="App\Jobs\JobGagal::dispatch();"

```
- [3] Amati percobaan ulang di log worker

```
docker compose logs horizon --tail 30

```
- [4] Lihat daftar job yang gagal permanen

```
docker compose exec app php artisan queue:failed

```
- [5] Jalankan ulang satu job gagal

```
docker compose exec app php artisan queue:retry <uuid>

```

Yang harus terlihat:

- Log menampilkan baris `JobGagal ... FAIL` beberapa kali berturut-turut.
- `queue:failed` menampilkan satu baris berisi uuid, nama job, dan waktu kegagalan.

Jawaban Acuan

L2.4.1 Berapa kali job dicoba sebelum dinyatakan gagal permanen?

Tiga kali, sesuai `tries` pada konfigurasi Horizon (`config/horizon.php`). Bila nilainya diubah, jumlah percobaan ikut berubah.

L2.4.2 Di mana catatan job gagal disimpan, dan bagaimana cara melihatnya?

Di tabel `failed_jobs` pada database. Dilihat dengan `php artisan queue:failed`, atau lewat tab Failed Jobs pada dashboard Horizon.

L2.4.3 Salin baris pesan kesalahan aslinya.

`RuntimeException: Sengaja gagal untuk latihan. — beserta jejak tumpukan yang menunjuk ke metode handle() pada JobGagal.`

L2.4.4 Bagaimana cara menjalankan ulang job yang sudah gagal?

`php artisan queue:retry <uuid>` untuk satu job, atau `queue:retry all` untuk semuanya.

Pembahasan

- Kesalahan umum: menyimpulkan job hanya dicoba sekali karena hanya membaca beberapa baris terakhir log. Percobaan ulang berjarak beberapa detik (`backoff`), jadi perlu menunggu.
- Kesalahan umum: mencari catatan kegagalan di Redis. Antreannya memang di Redis, tetapi catatan job GAGAL disimpan di database.
- Poin penting: peserta yang menyadari bahwa job gagal permanen TIDAK memunculkan error di sisi pengguna sama sekali — inilah alasan pemantauan antrean tidak bisa diabaikan.

L2.5 Cache Konfigurasi di Waktu yang Salah

Soal ringkas: Seorang rekan memindahkan `php artisan config:cache` dari entrypoint ke dalam Dockerfile dengan alasan "biar start-nya lebih cepat".

Konsep yang dilatih: Cache konfigurasi Laravel · Build time vs runtime · Sumber nilai environment

Langkah Penyelesaian

- [1] Bangun image dengan config:cache di dalam Dockerfile (versi bermasalah)
`RUN php artisan config:cache`
- [2] Jalankan container dengan DB_HOST yang jelas berbeda
`docker run --rm -e DB_HOST=host-yang-berbeda \`
`--entrypoint sh laravel-app:uji -c "php artisan tinker \`
`--execute=\"echo config('database.connections.mysql.host');\""`
- [3] Bandingkan dengan nilai environment yang sebenarnya diberikan
 (nilai config berbeda dengan nilai environment = dugaan terbukti)
- [4] Ulangi pada image yang config:cache-nya ada di entrypoint

Yang harus terlihat:

- Pada versi bermasalah, nilai config tetap memakai nilai saat build dan MENGABAIKAN environment yang diberikan saat run.
- Pada versi yang benar, nilai config mengikuti environment container.

Jawaban Acuan

L2.5.1 Apa yang sebenarnya disimpan `config:cache` ke dalam berkas?

Seluruh hasil evaluasi berkas config/ — termasuk setiap pemanggilan `env()` — dibekukan menjadi satu berkas PHP di `bootstrap/cache/config.php`. Sesudah cache itu ada, `env()` tidak lagi dibaca saat aplikasi berjalan.

L2.5.2 Kenapa nilai di server tidak terpakai?

Karena perintah dijalankan saat BUILD, yang terbaca adalah environment mesin build — bukan environment server. Nilai server tidak pernah ikut, dan tidak ada pesan error apa pun yang memberi tahu.

L2.5.3 Rancang langkah untuk membuktikan dugaan Anda.

Jalankan container dari image itu dengan DB_HOST yang sengaja dibuat berbeda, lalu bandingkan keluaran `config('database.connections.mysql.host')` dengan nilai environment yang diberikan. Bila keduanya berbeda, dugaan terbukti.

L2.5.4 Kapan `config:cache` seharusnya dijalankan, dan kenapa?

Saat RUNTIME, di entrypoint, setelah environment container tersedia. Dengan begitu yang ter-cache adalah nilai milik lingkungan tempat aplikasi benar-benar berjalan.

Pembahasan

- Kesalahan umum: menyalahkan berkas `.env` yang tidak ikut ke image. Justru sebaliknya — `.env` memang TIDAK boleh ikut; masalahnya ada pada waktu eksekusi `config:cache`.
- Kesalahan umum: menyarankan `config:clear` di entrypoint sebagai perbaikan. Itu memang menghilangkan gejalanya, tetapi juga membuang manfaat cache. Yang benar adalah memindahkan pembuatan cache-nya, bukan menghapusnya.
- Poin penting: peserta yang menyebut bahwa gejalanya SENYAP — tidak ada error, aplikasi hanya memakai nilai yang salah — menangkap inti bahayanya.

L2.6 Menguji Paritas Dua Web Server

Soal ringkas: Stack ini menyediakan dua web server yang dapat dipilih. Susun daftar periksa perilaku, lalu uji KEDUANYA dan buktikan apakah benar-benar setara.

Konsep yang dilatih: Paritas konfigurasi · Pengujian perilaku · Header keamanan

Langkah Penyelesaian

```
[1] Jalankan dengan Caddy (bawaan), lalu uji
    foreach ($p in "/", "/up", "/robots.txt", "/.htaccess", "/index.php") {
        "$p -> $(curl -s -o $null -w '%{http_code}' http://localhost:8080$p)" }

[2] Periksa header respons
    curl -s -D - -o $null http://localhost:8080/

[3] Pindah ke nginx: ubah COMPOSE_PROFILES=nginx di .env
    docker compose down --remove-orphans
    docker compose up -d

[4] Ulangi langkah [1] dan [2], lalu bandingkan
```

Yang harus terlihat:

- Kedua web server: halaman utama dan /up menjawab 200, /.htaccess menjawab 403.
- Kedua web server memasang X-Frame-Options, X-Content-Type-Options, dan Referrer-Policy.
- Satu-satunya perbedaan: akses langsung ke /index.php dijawab 308 oleh Caddy dan 404 oleh nginx — keduanya sama-sama menolak.

Jawaban Acuan

L2.6.1 Tulis daftar periksa yang Anda susun.

Daftar periksa yang memadai memuat: halaman utama, endpoint kesehatan, aset ber-hash, berkas unggahan di /storage, berkas titik yang benar-benar ada, akses langsung ke index.php, header keamanan, dan kompresi.

L2.6.2 Adakah perbedaan hasil antara kedua web server? Sebutkan.

Hanya satu: /index.php dijawab 308 (redirect) oleh Caddy dan 404 oleh nginx. Keduanya menolak akses langsung, hanya caranya berbeda. Sisanya identik.

L2.6.3 Header keamanan apa saja yang muncul pada respons halaman utama?

X-Frame-Options: SAMEORIGIN, X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin, dan X-XSS-Protection: 0.

L2.6.4 Kenapa menguji berkas titik yang TIDAK ADA bukan pengujian yang sah?

Berkas yang tidak ada dijawab 404 karena TIDAK DITEMUKAN — bukan karena ditolak aturan keamanan. Hasilnya tampak benar walaupun aturan penolakannya sebenarnya tidak bekerja. Pengujian sah memakai berkas yang benar-benar ada, mis. public/.htaccess.

Pembahasan

- Pertanyaan keempat adalah inti latihan ini. Persis kesalahan itu yang membuat sebuah kebocoran nyata pada konfigurasi Caddy tidak terdeteksi: /.htaccess tersaji dengan status 200 karena urutan direktif, dan pengujian dengan berkas fiktif tidak akan menemukannya.

- Kesalahan umum: lupa `--remove-orphans` saat berganti profil, sehingga web server lama tetap hidup, port bentrok, dan peserta menguji server yang salah.

L2.7 Mengukur Dampak Penempatan vendor/

Soal ringkas: Ukur waktu respons aplikasi dalam mode pengembangan dengan `vendor/` berada di bind mount, lalu ukur ulang setelah `vendor/` dipindah ke named volume.

Konsep yang dilatih: Kinerja bind mount · OPcache validate_timestamps · Named volume

Langkah Penyelesaian

```
[1] Pastikan vendor/ berada di bind mount, lalu panaskan aplikasi
    curl -s -o $null http://localhost:8080/

[2] Ukur tiga kali dengan jeda
    1..3 | ForEach-Object { Start-Sleep 4;
        Measure-Command { curl -s -o $null http://localhost:8080/ } }

[3] Pindahkan vendor/ ke named volume (compose.override.yaml)
    volumes:
      - ./src:/var/www/html
      - vendor:/var/www/html/vendor

[4] Terapkan dan ulangi pengukuran
    docker compose up -d
```

Yang harus terlihat:

- Dengan bind mount: sekitar 2 detik per request pada Docker Desktop Windows.
- Dengan named volume: sekitar 0,15 detik per request.

Jawaban Acuan

L2.7.1 Berapa waktu respons dengan vendor/ di bind mount?

Sekitar 2,0 detik pada stack acuan. Angka pastinya bergantung mesin, tetapi urutan besarnya adalah satuan detik.

L2.7.2 Berapa waktu respons dengan vendor/ di named volume?

Sekitar 0,15 detik — kira-kira 13 kali lebih cepat.

L2.7.3 Apa penyebab teknis perbedaan itu?

OPcache harus memeriksa ulang setiap berkas PHP untuk mendeteksi perubahan. Direktori `vendor/` berisi sekitar 10.000 berkas, dan setiap pemeriksaan berkas lewat bind mount Windows harus melintasi lapisan penerjemah antara Windows dan mesin virtual Linux. Ribuan operasi mahal itulah yang menumpuk menjadi detik.

L2.7.4 Apa konsekuensi memindahkan vendor/ ke named volume, dan bagaimana menanganinya?

Salinan `./src/vendor` di host tidak lagi ikut ter-update saat menambah paket, karena container memakai isi volume. Salinan host hanya dipakai IDE untuk autocomplete. Menanganinya: segarkan sesekali dengan `docker compose cp app:/var/www/html/vendor/. ./src/vendor`, atau pindahkan seluruh proyek ke filesystem WSL2 sehingga trik ini tidak diperlukan sama sekali.

Pembahasan

- Kesalahan umum: mengukur request pertama setelah container start. Saat itu OPcache masih dingin dan angkanya jauh lebih besar, sehingga perbandingannya tidak adil.
- Kesalahan umum: menyimpulkan "Docker di Windows memang lambat". Yang lambat spesifik: operasi metadata berkas lintas bind mount. CPU dan jaringan tidak terpengaruh.
- Diterima juga: peserta yang menyarankan menaikkan `opcache.revalidate_freq` sebagai solusi alternatif. Itu memang membantu, tetapi dampaknya jauh lebih kecil daripada memindahkan vendor/.

Hari 3 — Kubernetes: Konsep Inti

LATIHAN HARI 3

- L3.1 — Menelusuri Deployment sampai Endpoint
- L3.2 — Perubahan Konfigurasi yang Tidak Berlaku
- L3.3 — Menguji Ketahanan Data
- L3.4 — Memilih Probe yang Tepat
- L3.5 — Membuat Overlay Staging

L3.1 Menelusuri Deployment sampai Endpoint

Soal ringkas: Mulai dari satu Deployment, telusuri seluruh rantai objek sampai ke alamat IP pod yang benar-benar menerima trafik.

Konsep yang dilatih: Deployment · ReplicaSet · Service · EndpointSlice · Pemilihan lewat label

Langkah Penyelesaian

- [1] Lihat rantai objeknya
`kubectl get deploy,rs,pod -n laravel -l app.kubernetes.io/name=app`
- [2] Lihat pod yang terdaftar sebagai endpoint
`kubectl get endpointslice -n laravel`
`kubectl get pod -n laravel -o wide -l app.kubernetes.io/name=app`
- [3] Hapus satu pod, amati penggantinya
`kubectl delete pod <nama-pod> -n laravel`
`kubectl get pod -n laravel -o wide -w`
- [4] Periksa ulang endpoint
`kubectl get endpointslice -n laravel`

Yang harus terlihat:

- Deployment app -> ReplicaSet app-<hash> -> dua Pod app-<hash>-<acak>.
- Pod pengganti punya nama DAN alamat IP yang berbeda.
- EndpointSlice ikut diperbarui secara otomatis.

Jawaban Acuan

L3.1.1 Tulis rantai objeknya lengkap dengan nama masing-masing.

deployment/app -> replicaset/app-<hash> -> pod/app-<hash>-<acak> (dua buah). Service `app` memilih pod-pod itu lewat label `app.kubernetes.io/name=app`.

L3.1.2 Alamat IP apa saja yang terdaftar di EndpointSlice sebelum penghapusan?

Dua alamat IP internal cluster, mis. 10.1.0.x. Angkanya berbeda-beda tiap cluster.

L3.1.3 Setelah satu pod dihapus, apa yang berubah pada nama pod dan alamat IP-nya?

Nama pod baru berbeda dari yang dihapus (bagian acaknya berubah), dan alamat IP-nya juga baru. EndpointSlice diperbarui otomatis mengikutinya.

L3.1.4 Kenapa web server tetap bisa menghubungi aplikasi walau alamat IP berubah?

Karena web server memanggil NAMA Service (`app`), bukan alamat IP pod. Service menjaga alamat tetap dan meneruskan trafik ke pod mana pun yang sedang terdaftar sebagai endpoint. Inilah alasan pod boleh bersifat fana.

Pembahasan

- Kesalahan umum: mengira Service menyimpan daftar alamat IP secara manual. Pemilihannya dinamis berdasarkan label; pod baru yang berlabel cocok otomatis ikut terdaftar.
- Poin penting: peserta yang menyadari pod hanya masuk EndpointSlice setelah lolos `readinessProbe` menghubungkan latihan ini dengan L3.4.

L3.2 Perubahan Konfigurasi yang Tidak Berlaku

Soal ringkas: Ubah satu nilai konfigurasi non-rahasia, terapkan, lalu buktikan apakah pod benar-benar memakai nilai yang baru.

Konsep yang dilatih: ConfigMap · configMapGenerator · Pemicu rollout · Immutability lewat hash

Langkah Penyelesaian

```
[1] Lihat nama ConfigMap saat ini
    kubectl get configmap -n laravel

[2] Ubah satu nilai di k8s/base/app-config.env, mis. LOG_LEVEL

[3] Terapkan dan amati
    kubectl apply -k k8s/overlays/local
    kubectl get configmap -n laravel
    kubectl get pod -n laravel -w

[4] Periksa nilai yang benar-benar diterima proses
    kubectl exec -n laravel deploy/app -- printenv LOG_LEVEL
```

Yang harus terlihat:

- Nama ConfigMap berubah karena imbuhan hash-nya ikut berubah.
- Pod otomatis di-rollout tanpa perintah tambahan.
- `printenv` menampilkan nilai yang baru.

Jawaban Acuan

L3.2.1 Dengan ConfigMap biasa: apakah nilai di dalam pod ikut berubah? Kenapa?

TIDAK berubah. ConfigMap di cluster memang sudah berisi nilai baru, tetapi variabel environment hanya dibaca sekali saat container dibuat. Pod yang sedang berjalan tetap memegang nilai lama sampai ada yang me-restart-nya.

L3.2.2 Dengan configMapGenerator: apa yang berubah pada nama ConfigMap?

Imbuan hash pada namanya berubah, mis. dari app-config-mbgkf6m4hc menjadi nama lain, karena hash dihitung dari isi berkas.

L3.2.3 Kenapa perubahan nama memicu rollout?

Nama ConfigMap tercantum di dalam spesifikasi pod pada Deployment. Nama berubah berarti spesifikasi berubah, dan spesifikasi yang berubah otomatis memicu rolling update.

L3.2.4 Kenapa pendekatan pertama berbahaya di produksi?

Karena gejalanya senyap. Manifest terlihat benar, `kubectl apply` berhasil, dan `kubectl get configmap` menampilkan nilai baru — tetapi aplikasi tetap memakai nilai lama. Tidak ada satu pun pesan yang memberi tahu, dan kesalahan seperti ini bisa bertahan berminggu-minggu tanpa disadari.

Pembahasan

- Kesalahan umum: menyimpulkan konfigurasi berhasil hanya dengan `kubectl get configmap`. Itu memeriksa keadaan cluster, bukan keadaan proses di dalam pod.
- Diterima juga: peserta yang menyebut `kubectl rollout restart` sebagai penyelesaian manual, asalkan ia juga menjelaskan kenapa cara otomatis lebih dapat diandalkan.
- Catatan: ConfigMap yang dipasang sebagai VOLUME memang ikut ter-update tanpa restart, tetapi dengan jeda, dan aplikasi tetap harus membaca ulang berkasnya. Peserta yang menyebut perbedaan ini layak dihargai.

L3.3 Menguji Ketahanan Data

Soal ringkas: Buktikan secara mandiri mana data yang bertahan dan mana yang hilang ketika pod dihapus: data MySQL, data Redis queue, data Redis cache, dan berkas unggahan.

Konsep yang dilatih: PersistentVolumeClaim · emptyDir · StatefulSet · Ketahanan data

Langkah Penyelesaian

- [1] Tulis penanda ke masing-masing penyimpanan


```
kubectl exec -n laravel deploy/app -- php artisan tinker \
  --execute="Cache::put('penanda','ada',3600);"
kubectl exec -n laravel deploy/app -- sh -c \
  "echo penanda > storage/app/public/uji.txt"
```
- [2] Catat keadaan awal


```
kubectl exec -n laravel deploy/app -- php artisan migrate:status
```
- [3] Hapus pod satu per satu, tunggu penggantinya siap


```
kubectl delete pod mysql-0 -n laravel
kubectl wait --for=condition=ready pod/mysql-0 -n laravel --timeout=300s
kubectl delete pod -n laravel -l app.kubernetes.io/name=redis-cache
```
- [4] Periksa ulang seluruh penanda
- [5] Lihat jenis volume tiap workload


```
kubectl get pod mysql-0 -n laravel -o jsonpath="{.spec.volumes}"
```

Yang harus terlihat:

- Migrasi MySQL tetap tercatat Ran setelah pod dihapus.
- Penanda cache HILANG setelah pod redis-cache diganti.
- Berkas unggahan tetap ada dan tetap dapat diakses lewat web server.

Jawaban Acuan

L3.3.1 Tulis tabel: penyimpanan, jenis volume, bertahan atau hilang.

MySQL — PVC (volumeClaimTemplate) — bertahan.

Redis queue — PVC (volumeClaimTemplate) — bertahan.
 Redis cache — emptyDir — hilang.
 Berkas unggahan — PVC ReadWriteMany — bertahan.

L3.3.2 Kenapa data Redis cache berperilaku berbeda dari Redis queue?

Karena keduanya sengaja dikonfigurasi berbeda. Redis queue memakai PVC dan AOF sebab job yang mengantre tidak boleh hilang. Redis cache memakai emptyDir sebab isinya bisa dibangun ulang kapan saja dari sumber aslinya, sehingga menyimpannya justru pemborosan.

L3.3.3 Apa yang terjadi pada berkas unggahan, dan kenapa?

Tetap ada, karena berada di PVC ReadWriteMany yang dipasang bersama oleh pod aplikasi dan pod web server. Volume itu hidup lebih lama daripada pod mana pun yang memakainya.

L3.3.4 Bagaimana Anda mengetahui jenis volume yang dipakai tiap workload?

Dengan `kubectl get pod <nama> -o jsonpath="{.spec.volumes}"`, atau membaca manifest-nya langsung. Volume berjenis persistentVolumeClaim bertahan; emptyDir tidak.

Pembahasan

- Kesalahan umum: memeriksa terlalu cepat, sebelum pod pengganti Ready, lalu menyimpulkan data hilang padahal database belum selesai start.
- Kesalahan umum: menganggap emptyDir sebagai kekurangan. Untuk cache, itu justru pilihan yang tepat dan disengaja.
- Poin penting: peserta yang menghubungkan hasil ini dengan L2.3 — kenapa cache dan queue dipisah — menunjukkan pemahaman yang menyatu.

L3.4 Memilih Probe yang Tepat

Soal ringkas: Untuk setiap skenario di bawah, tentukan probe mana yang tepat dan sebutkan setelan pentingnya. Jelaskan juga apa yang terjadi bila salah pilih.

Konsep yang dilatih: startupProbe · readinessProbe · livenessProbe · Kegagalan berantai

Langkah Penyelesaian

- [1] Lihat setelan probe yang sedang berlaku
`kubectl describe pod -n laravel -l app.kubernetes.io/name=app | \`
`Select-String "Probe"`
- [2] Amati readiness bekerja saat rollout
`kubectl rollout restart deployment/app -n laravel`
`kubectl get pod -n laravel -w`
- [3] Perhatikan pod baru: Running tetapi 0/1 sebelum lolos readiness

```
[4] Periksa kapan pod masuk daftar endpoint
kubectrl get endpointslice -n laravel
```

Yang harus terlihat:

- Pod baru berstatus Running tetapi 0/1 selama beberapa saat.
- Pod baru masuk EndpointSlice hanya setelah menjadi 1/1.

Jawaban Acuan

L3.4.1 Skenario A — probe apa, setelan apa, dan apa akibatnya bila salah?

startupProbe, dengan failureThreshold × periodSeconds yang melebihi 90 detik (mis. periodSeconds 3 dan failureThreshold 40 = 120 detik). Bila hanya memakai livenessProbe tanpa startupProbe, container dibunuh sebelum sempat siap dan terjebak dalam CrashLoopBackOff yang tak berujung.

L3.4.2 Skenario B — probe apa, dan kenapa BUKAN livenessProbe yang diperketat?

readinessProbe. Aplikasi yang sedang sibuk cukup DIKELUARKAN sementara dari Service, bukan dibunuh. Memperketat livenessProbe justru berbahaya: pod yang sibuk akan dibunuh, bebannya berpindah ke pod tersisa, pod itu ikut kewalahan lalu dibunuh juga — kegagalan berantai yang dipicu oleh mekanisme yang seharusnya menolong.

L3.4.3 Skenario C — probe apa, dan kenapa probe lain tidak menyelesaikannya?

livenessProbe. Aplikasi yang menggantung tidak akan pulih sendiri; satu-satunya jalan keluar adalah dimulai ulang. readinessProbe hanya mengeluarkannya dari Service dan membiarkannya menggantung selamanya, sementara startupProbe sudah tidak berlaku lagi setelah masa boot lewat.

L3.4.4 Kenapa livenessProbe sebaiknya tidak menyentuh database?

Karena database yang sedang lambat akan membuat probe gagal di SELURUH pod aplikasi secara bersamaan, sehingga semuanya dibunuh berbarengan. Masalah pada satu komponen berubah menjadi pemadaman total. livenessProbe sebaiknya hanya menguji kesehatan proses itu sendiri.

Pembahasan

- Kesalahan umum: memakai satu probe yang sama untuk ketiganya dengan alasan penyederhanaan. Akibatnya salah satu dari dua kegagalan pada skenario A atau B pasti terjadi.
- Cara menilai: jawaban lengkap harus menyebut probe DAN alasannya. Menyebut nama probe yang benar tanpa alasan bernilai separuh.
- Poin penting: peserta yang menyadari pertanyaan keempat adalah tentang radius kerusakan, bukan tentang benar-salah teknis, menangkap inti keandalan sistem.

L3.5 Membuat Overlay Staging

Soal ringkas: Buat overlay baru bernama `staging` yang berbeda dari overlay lokal dalam tiga hal: namespace `laravel-staging`, jumlah replika aplikasi 1, dan LOG_LEVEL `debug`.

Konsep yang dilatih: Kustomize overlay · Patch · Pemisahan lingkungan

Langkah Penyelesaian

```
[1] Buat direktori dan berkas overlay
k8s/overlays/staging/kustomization.yaml
```

- ```
[2] Isi berkasnya (lihat Jawaban Acuan)

[3] Periksa hasil render TANPA menerapkan
 kubectl kustomize k8s/overlays/staging

[4] Pastikan namespace, replika, dan LOG_LEVEL sudah sesuai
 kubectl kustomize k8s/overlays/staging | \
 Select-String "namespace:|replicas:|LOG_LEVEL"
```

Yang harus terlihat:

- Seluruh objek berada di namespace laravel-staging.
- Deployment app memiliki replicas: 1.
- LOG\_LEVEL bernilai debug.

## Jawaban Acuan

### L3.5.1 Tulis isi kustomization.yaml overlay staging Anda.

Isi yang benar memuat `namespace: laravel-staging`, `resources: [../../base]`, `secretGenerator` sendiri, satu patch untuk replicas, dan satu `configMapGenerator` dengan `behavior: merge` untuk menimpa LOG\_LEVEL:

```
namespace: laravel-staging
resources: [../../base]
configMapGenerator:
 - name: app-config
 behavior: merge
 literals: [LOG_LEVEL=debug]
patches:
 - target: { kind: Deployment, name: app }
 patch: |-
 - op: replace
 path: /spec/replicas
 value: 1
```

### L3.5.2 Bagaimana Anda mengubah LOG\_LEVEL tanpa menyentuh base?

Dengan `configMapGenerator` ber-`behavior: merge`, yang menimpa satu kunci saja tanpa menyalin ulang seluruh berkas konfigurasi base.

### L3.5.3 Perintah apa yang Anda pakai untuk memeriksa hasilnya sebelum menerapkan?

`kubectl kustomize k8s/overlays/staging`. Bila overlay sudah pernah diterapkan, `kubectl diff -k` memperlihatkan perubahannya saja.

### L3.5.4 Kenapa base sebaiknya tidak pernah diubah untuk kebutuhan satu lingkungan?

Karena base adalah sumber kebenaran bersama seluruh lingkungan. Menyisipkan kebutuhan satu lingkungan ke dalamnya membuat lingkungan lain ikut terpengaruh, dan cepat atau lambat base berisi campuran setelan yang tidak berlaku di mana-mana.

## Pembahasan

- Kesalahan umum: menyalin seluruh berkas base ke dalam overlay lalu mengeditnya. Hasilnya berfungsi, tetapi menghilangkan seluruh manfaat Kustomize dan setiap perbaikan harus dikerjakan dua kali.
- Kesalahan umum: lupa membuat `secrets.env` sendiri untuk staging, sehingga generator gagal.



- Diterima juga: memakai `replicas`: field bawaan Kustomize sebagai ganti patch JSON. Keduanya benar.

## Hari 4 — Produksi dan Keamanan

### LATIHAN HARI 4

- L4.1 — Memperbaiki Manifest yang Ditolak
- L4.2 — Menemukan Direktori yang Perlu Writable
- L4.3 — Menguji Apakah NetworkPolicy Benar-benar Ditegakkan
- L4.4 — Mengukur Kegagalan Request Selama Rollout
- L4.5 — Analisis Akar Masalah dari Log

### L4.1 Memperbaiki Manifest yang Ditolak

Soal ringkas: Manifest berikut ditolak oleh namespace berlabel Pod Security restricted.

Konsep yang dilatih: Pod Security Admission · securityContext · Membaca pesan penolakan

Kode yang diberikan di lembar soal

```
apiVersion: v1
kind: Pod
metadata:
 name: latihan-pss
 namespace: laravel
spec:
 containers:
 - name: alat
 image: busybox
 command: ["sh", "-c", "sleep 300"]
```

### Langkah Penyelesaian

- [1] Simpan manifest bermasalah lalu coba terapkan  
`kubectl apply -f latihan-pss.yaml`
- [2] Baca pesan penolakan dengan teliti – tiap pelanggaran disebutkan
- [3] Tambahkan securityContext di tingkat pod DAN container (lihat Jawaban Acuan)
- [4] Terapkan ulang dan pastikan pod berjalan  
`kubectl apply -f latihan-pss.yaml`  
`kubectl get pod latihan-pss -n laravel`
- [5] Bersihkan  
`kubectl delete pod latihan-pss -n laravel`

Yang harus terlihat:

- Penolakan menyebut empat pelanggaran sekaligus.
- Setelah diperbaiki, pod berstatus Running.

### Jawaban Acuan

#### L4.1.1 Salin pesan penolakan yang Anda terima.

Error from server (Forbidden): pods "latihan-pss" is forbidden: violates PodSecurity "restricted:latest": allowPrivilegeEscalation != false, unrestricted capabilities, runAsNonRoot != true, seccompProfile.

#### L4.1.2 Ada berapa pelanggaran, dan apa saja?

Empat: allowPrivilegeEscalation belum false; capabilities belum di-drop; runAsNonRoot belum true; seccompProfile belum diset.

#### L4.1.3 Tulis manifest yang sudah diperbaiki.

spec:

```
securityContext:
 runAsNonRoot: true
 runAsUser: 1000
 seccompProfile: { type: RuntimeDefault }
containers:
- name: alat
 image: busybox
 command: ["sh", "-c", "sleep 300"]
 securityContext:
 allowPrivilegeEscalation: false
 capabilities: { drop: ["ALL"] }
```

#### L4.1.4 Kenapa menurunkan label namespace menjadi `baseline` bukan penyelesaian yang benar?

Karena yang bermasalah adalah manifest-nya, bukan kebijakannya. Menurunkan label melonggarkan aturan untuk SELURUH beban kerja di namespace itu — termasuk yang sudah benar — demi satu pod yang belum disesuaikan. Kebijakan keamanan yang dilonggarkan karena satu kasus jarang dikembalikan lagi.

### Pembahasan

- Kesalahan umum: menaruh seluruh setelan di tingkat container saja. `runAsNonRoot` dan `seccompProfile` boleh di kedua tingkat, tetapi menaruhnya di tingkat pod lebih rapi karena berlaku untuk semua container.
- Kesalahan umum: menyetel `runAsUser: 0`. Itu tetap root dan tetap ditolak.
- Diterima juga: `runAsUser` dengan angka lain selain 1000, asalkan bukan 0.

## L4.2 Menemukan Direktori yang Perlu Writable

Soal ringkas: Sebuah container dijalankan dengan `readOnlyRootFilesystem: true` dan gagal.

Konsep yang dilatih: `readOnlyRootFilesystem` · `emptyDir` · Hak akses minimal

### Langkah Penyelesaian

```
[1] Buktikan rootfs memang read-only
kubect exec -n laravel deploy/app -- sh -c "touch /coba"

[2] Uji tiap direktori yang dicurigai
kubect exec -n laravel deploy/app -- sh -c \
 "touch storage/logs/uji && echo BISA || echo TIDAK BISA"
kubect exec -n laravel deploy/app -- sh -c \
 "touch bootstrap/cache/uji && echo BISA || echo TIDAK BISA"
kubect exec -n laravel deploy/app -- sh -c \
 "touch /tmp/uji && echo BISA || echo TIDAK BISA"

[3] Uji perkakas yang menulis ke direktori home
kubect exec -n laravel deploy/app -- php artisan tinker --execute="echo 1;"
```

```
[4] Lihat volume yang sudah dipasang
kubect1 get pod -n laravel -l app.kubernetes.io/name=app \
-o jsonpath="{.items[0].spec.containers[0].volumeMounts}"
```

Yang harus terlihat:

- Langkah [1] ditolak dengan "Read-only file system".
- Ketiga direktori pada langkah [2] menjawab BISA karena sudah dipasang sebagai volume.

## Jawaban Acuan

### L4.2.1 Direktori apa saja yang perlu writable? Sebutkan beserta alasannya.

`/var/www/html/storage` (log, cache framework, sesi, berkas unggahan) — PVC agar bertahan;  
`/var/www/html/bootstrap/cache` (cache config, route, view yang dibangun entrypt) —  
 emptyDir cukup karena dibangun ulang tiap start; `/tmp` (berkas sementara PHP dan perkakas) —  
 emptyDir.

### L4.2.2 Bagaimana Anda membuktikan sebuah direktori benar-benar tidak bisa ditulis?

Dengan mencoba menulis ke dalamnya, mis. `touch <path>` lewat `kubect1 exec`, dan melihat apakah muncul "Read-only file system".

### L4.2.3 Perkakas apa yang gagal karena direktori home tidak writable, dan bagaimana solusinya?

`php artisan tinker` gagal dengan pesan "Writing to directory /home/www-data/.config/psysch is not allowed". Solusinya menyetel `HOME=/tmp` lewat ConfigMap, karena `/tmp` sudah tersedia sebagai emptyDir yang writable.

### L4.2.4 Kenapa mematikan readOnlyRootFilesystem bukan penyelesaian yang benar?

Karena itu membuang lapisan pertahanan demi kenyamanan. Dengan rootfs writable, penyerang yang berhasil masuk dapat menanam berkas, mengubah kode aplikasi, atau memasang perkakas tambahan. Menemukan tiga direktori yang perlu ditulis jauh lebih murah daripada kehilangan seluruh lapisan itu.

## Pembahasan

- Kesalahan umum: memasang seluruh `/var/www/html` sebagai emptyDir. Kode aplikasi ikut tertimpa direktori kosong dan aplikasi tidak bisa jalan sama sekali.
- Kesalahan umum: memakai PVC untuk bootstrap/cache. Tidak perlu — isinya dibangun ulang setiap start, dan memakai PVC justru menambah ketergantungan yang tidak berguna.
- Poin penting: peserta yang menemukan masalah HOME secara mandiri lewat pengujian, bukan dari petunjuk, menunjukkan cara kerja diagnosis yang benar.

## L4.3 Menguji Apakah NetworkPolicy Benar-benar Ditegakkan

Soal ringkas: Seluruh NetworkPolicy sudah diterapkan tanpa error. Jangan percaya begitu saja.

Konsep yang dilatih: NetworkPolicy · CNI · Verifikasi klaim keamanan

## Langkah Penyelesaian

```
[1] Pastikan kebijakan memang sudah ada
kubect1 get networkpolicy -n laravel
```

```
[2] Jalankan pod uji TANPA label yang diizinkan mana pun
```

```
kubectl run netpol-uji -n laravel --image=redis:8-alpine --restart=Never \
 --overrides='{ "spec": { "securityContext": { "runAsNonRoot": true,
 "runAsUser": 999, "seccompProfile": { "type": "RuntimeDefault" } } } }' \
 --command -- sleep 120
```

```
[3] Coba hubungi penyimpanan dari pod itu
kubectl exec -n laravel netpol-uji -- nc -z mysql 3306
kubectl exec -n laravel netpol-uji -- nc -z redis 6379
 nc -z = uji apakah port bisa dihubungi, tanpa mengirim data
```

```
[4] Bersihkan
kubectl delete pod netpol-uji -n laravel
```

Yang harus terlihat:

- DI DOCKER DESKTOP: kedua koneksi BERHASIL — membuktikan kebijakan tidak ditegakkan.
- Di cluster dengan Calico atau Cilium: kedua koneksi gagal.

## Jawaban Acuan

### L4.3.1 Bagaimana rancangan pengujian Anda?

Membuat pod di namespace yang sama, TANPA label yang diizinkan kebijakan mana pun, lalu mencoba membuka koneksi ke MySQL dan Redis dari pod itu. Bila kebijakan ditegakkan, keduanya harus gagal.

### L4.3.2 Apa hasilnya di Docker Desktop?

Kedua koneksi BERHASIL. Kebijakan tidak ditegakkan sama sekali.

### L4.3.3 Apa kesimpulan Anda tentang postur keamanan cluster ini?

Di Docker Desktop, setiap pod di cluster dapat menghubungi MySQL dan Redis secara langsung. Satu-satunya lapisan yang tersisa adalah password. Manifest-nya sendiri benar dan akan bekerja di cluster ber-CNI yang mendukung, tetapi di sini ia tidak melindungi apa pun.

### L4.3.4 Kenapa manifest yang ter-apply tanpa error tidak membuktikan apa pun?

Karena API server hanya memvalidasi BENTUK objeknya, bukan apakah ada komponen yang akan menjalankannya. NetworkPolicy diterapkan oleh CNI; bila CNI cluster tidak mengimplementasikannya, objek tetap tersimpan rapi dan tidak melakukan apa-apa. Tidak ada peringatan apa pun — dan keamanan yang terlihat ada tetapi tidak bekerja lebih berbahaya daripada tidak ada sama sekali, karena membuat orang berhenti waspada.

## Pembahasan

- Kesalahan umum: menguji dari pod aplikasi yang justru DIIZINKAN oleh kebijakan. Koneksinya berhasil, dan peserta salah menyimpulkan bahwa kebijakan bekerja.
- Kesalahan umum: lupa memberi securityContext pada pod uji, sehingga ditolak Pod Security dan peserta mengira itu bukti NetworkPolicy bekerja.
- Cara menilai: yang paling penting adalah pertanyaan keempat. Peserta yang menyimpulkan "harus diuji ulang di setiap cluster baru" sudah menangkap pelajaran utama modul ini.

## L4.4 Mengukur Kegagalan Request Selama Rollout

Soal ringkas: Ukur secara kuantitatif berapa request yang gagal selama rolling update.

Konsep yang dilatih: Rolling update · maxUnavailable · preStop · Pengukuran keandalan

## Langkah Penyelesaian

```
[1] Jalankan pemantau di satu jendela
1..40 | ForEach-Object {
 "$(curl -s -o $null -w '%{http_code}' http://localhost/up)"
 Start-Sleep -Milliseconds 500 }

[2] Di jendela lain, lakukan rollout
kubectl rollout restart deployment/app -n laravel
kubectl rollout status deployment/app -n laravel

[3] Hitung kode selain 200

[4] Ubah maxUnavailable menjadi 2 dan hapus readinessProbe, lalu ulangi
kubectl patch deployment app -n laravel --type=json -p \
'[{ "op": "replace", "path": "/spec/strategy/rollingUpdate/maxUnavailable",
 "value": 2 }]'

[5] Kembalikan ke konfigurasi semula
kubectl apply -k k8s/overlays/local
```

Yang harus terlihat:

- Konfigurasi bawaan: seluruh request menjawab 200.
- Setelah maxUnavailable dinaikkan dan readiness dilemahkan: muncul sejumlah 502 atau 503.

## Jawaban Acuan

### L4.4.1 Berapa request yang gagal pada konfigurasi bawaan? Dari berapa total?

Nol dari total request yang dikirim. Pada stack acuan: 30 request, 0 gagal.

### L4.4.2 Setelan apa yang Anda ubah untuk membuat rollout menyebabkan kegagalan?

Menaikkan `maxUnavailable` sehingga kedua pod boleh berhenti bersamaan, dan/atau melemahkan `readinessProbe` sehingga pod dianggap siap sebelum benar-benar siap.

### L4.4.3 Berapa request yang gagal setelah perubahan itu?

Bergantung besar perubahan; yang penting jumlahnya jelas lebih dari nol dan peserta benar-benar menghitungnya, bukan menebak.

### L4.4.4 Setelan mana yang paling menentukan, dan kenapa?

`maxUnavailable: 0` yang paling menentukan, karena ia menjamin kapasitas tidak pernah turun di bawah jumlah yang diminta. Tetapi ia hanya bermakna bila `readinessProbe` benar — sebab "tersedia" didefinisikan oleh `readiness`. Keduanya bekerja berpasangan: `maxUnavailable` menentukan aturannya, `readinessProbe` menentukan artinya.

## Pembahasan

- Kesalahan umum: memantau halaman utama yang berat alih-alih endpoint kesehatan, sehingga hasilnya bercampur dengan lambatnya render.
- Kesalahan umum: menjalankan rollout sebelum pemantau siap, sehingga jendela kegagalannya terlewat.
- Poin penting: peserta yang menyadari bahwa `maxUnavailable` tanpa `readinessProbe` yang benar hanyalah janji kosong — pod dihitung tersedia padahal belum siap — menangkap hubungan yang paling sering luput.

## L4.5 Analisis Akar Masalah dari Log

Soal ringkas: Sebuah stack Kubernetes menunjukkan gejala berikut. Database berstatus Running dan sehat, tetapi aplikasi ditolak dengan pesan:

Konsep yang dilatih: Diagnosis akar masalah · Log container sebelumnya · Efek samping environment

### Langkah Penyelesaian

```
[1] Periksa status dan jumlah restart
 kubectl get pods -n laravel

[2] Baca Events dan penyebab terminasi sebelumnya
 kubectl describe pod mysql-0 -n laravel
 kubectl get pod mysql-0 -n laravel \
 -o jsonpath="{.status.containerStatuses[0].lastState}"

[3] Baca log container yang SEKARANG – perhatikan yang TIDAK ada
 kubectl logs mysql-0 -n laravel | Select-String "Initializing database"

[4] Baca log container SEBELUM restart – di sinilah jawabannya
 kubectl logs mysql-0 -n laravel --previous

[5] Periksa environment yang disuntikkan
 kubectl get statefulset mysql -n laravel \
 -o jsonpath="{.spec.template.spec.containers[0].env[*].name}"
```

Yang harus terlihat:

- Log container sekarang TIDAK memuat "Initializing database files".
- Log `--previous` memuat: Temporary server started, lalu ERROR 1045 Access denied for user 'root'@'localhost'.
- Daftar environment memuat MYSQL\_PWD.

### Jawaban Acuan

#### L4.5.1 Perintah apa saja yang Anda jalankan, berurutan?

`kubectl get pods`; `kubectl describe pod`; membaca `lastState`; `kubectl logs`; lalu — yang menentukan — `kubectl logs --previous`; dan terakhir memeriksa daftar environment `StatefulSet`.

#### L4.5.2 Apa yang Anda temukan di log container SEBELUM restart terakhir?

Inisialisasi berjalan sampai "Temporary server started", lalu berhenti dengan ERROR 1045 Access denied for user 'root'@'localhost' (using password: YES), dan container keluar dengan status 1.

#### L4.5.3 Apa akar masalahnya?

Variabel `MYSQL_PWD` disetel di tingkat container agar password tidak muncul di daftar proses. Tetapi klien `mysql` membaca variabel itu SECARA OTOMATIS — termasuk ketika entrypoint resmi MySQL menyiapkan database lewat temporary server yang root-nya masih berpassword KOSONG. Password yang tidak diminta itu membuat autentikasi ditolak, inisialisasi gagal di tengah, dan direktori data tertinggal berisi tabel sistem tanpa user aplikasi.

#### L4.5.4 Kenapa menghapus PVC tidak menyelesaikannya, dan apa perbaikan permanennya?

Karena provisioner `hostpath` Docker Desktop memetakan volume berdasarkan NAMA PVC. Menghapus PVC tidak menghapus isi direktori di host, sehingga PVC baru bernama sama menemukan data rusak yang lama. Perbaikan permanennya ada dua dan keduanya diperlukan: (a)

ganti nama variabel menjadi mis. APP\_DB\_PASSWORD dan setel MYSQL\_PWD hanya DI DALAM perintah probe, dan (b) kosongkan direktori data secara eksplisit sebelum menjalankan ulang.

### Pembahasan

- Kesalahan umum: berhenti di `kubectl logs` tanpa `--previous`. Log container yang sekarang terlihat normal justru karena inisialisasi DILEWATI — bukti kuncinya hanya ada di log container sebelumnya.
- Kesalahan umum: menyalahkan grant MySQL atau NetworkPolicy karena pesan errornya menyebut host. Pesan itu gejala, bukan penyebab.
- Diterima juga: peserta yang menemukan akar masalah lewat jalur berbeda, mis. memeriksa isi tabel `mysql.user` dan mendapati user aplikasi tidak ada. Kesimpulannya sama.
- Cara menilai: pertanyaan keempat bernilai paling tinggi. Jawaban lengkap harus menyebut DUA sebab — perilaku provisioner hostpath DAN efek samping MYSQL\_PWD. Menyebut satu saja berarti masalahnya akan terulang.

## Penutup

### TIGA HAL YANG PALING LAYAK DITEKANKAN DI AKHIR

Konfigurasi yang ter-apply tanpa error belum tentu bekerja. L2.6 dan L4.3 dirancang khusus untuk menanamkan kebiasaan menguji perilaku, bukan membaca ulang konfigurasi.

Pesan error sering menunjuk ke gejala, bukan penyebab. L4.5 melatih peserta menelusuri sampai ke log container sebelumnya.

Kegagalan yang meninggalkan keadaan setengah jadi lebih berbahaya daripada kegagalan total. L2.2 dan L4.5 dua-duanya bermuara ke sana.